

Task-Specific Sub-Agents with NemoClaw, OpenClaw, OpenWebUI, and Ollama

This setup uses OpenClaw sub-agents inside a NemoClaw sandbox while using Ollama as the local model backend. OpenWebUI can also connect to the same Ollama server for normal chat, but OpenWebUI does not control OpenClaw sub-agents directly.

Recommended Architecture

``text id="agxdrx" OpenWebUI -> Ollama -> local model

NemoClaw / OpenClaw sandbox -> Ollama or OpenAI-compatible Ollama endpoint -> primary agent model -> optional sub-agent model

OpenWebUI is the chat UI.

Ollama is the inference backend.

OpenClaw/NemoClaw manages the agent and sub-agent behavior.

Important Difference from OpenWebUI

OpenClaw sub-agents use OpenClaw features such as:

```
``text id="mdnsw6"
sessions_spawn
agents.list
agents.defaults
models.providers
per-agent auth profiles
tool policy
```

OpenWebUI does not automatically use these settings.

So this works inside NemoClaw/OpenClaw:

``text id="n772ng" Primary agent -> sessions_spawn -> task-specific sub-agent

But OpenWebUI does not automatically create or route to these OpenClaw sub-agents.

NemoClaw Sandbox Paths

Inside the NemoClaw sandbox, use these paths:

```
```text id="glg3to"
/sandbox/.openclaw/openclaw.json
```

Main OpenClaw configuration. This includes:

```
```text id="ymmn57" models.providers agents.defaults agents.list
```

```
```text id="q19uj8"
/sandbox/.openclaw/.config-hash
```

Hash for `openclaw.json`. Update it after manually editing the config.

```
```text id="jztgvc" /sandbox/.openclaw/agents/<agent-id>/agent/auth-profiles.json
```

Per-agent credentials. Use this only if a sub-agent calls a different provider directly.

```
```text id="n9me09"
/sandbox/.openclaw/workspace/
```

Writable shared workspace for files passed between the primary agent and sub-agents.

```
```text id="guxqrn" /tmp/gateway.log
```

Gateway log for checking config reloads and sub-agent failures.

For file-based tasks, tell agents to use:

```
```text id="w1ha2z"
/sandbox/.openclaw/workspace/
```

Avoid older `.openclaw-data` paths or read-only OpenClaw directories.

## Ollama-Based Sub-Agent Example

Instead of an Omni VLM provider, you can define a task-specific sub-agent backed by an Ollama model.

Example:

```
```text id="0p11tu" Primary agent: main Primary model: ollama/qwen2.5:14b Sub-agent: code-operator
Sub-agent model: ollama/qwen2.5-coder:7b Delegation tool: sessions_spawn Backend: Ollama
```

The primary agent handles planning and conversation.

The sub-agent handles a specific task, such as code review, log analysis, or command generation.

Example Provider Shape for Ollama

Your OpenClaw config needs a provider that points to Ollama.

If OpenClaw supports Ollama directly, use the native Ollama endpoint:

```
```text id="3ft5q3"
http://localhost:11434
```

If OpenClaw expects an OpenAI-compatible API, use Ollama's OpenAI-compatible endpoint:

```
```text id="0kn91k" http://localhost:11434/v1
```

Example conceptual provider:

```
```json id="u7fitk"
{
 "models": {
 "providers": {
 "ollama-local": {
 "type": "openai-compatible",
 "baseUrl": "http://localhost:11434/v1",
 "apiKey": "dummy"
 }
 }
 }
}
```

Use a non-empty placeholder API key such as:

```
```text id="jaqv7k" dummy
```

if Ollama does not require authentication.

Example agents.list Shape

A simplified conceptual config may look like this:

```
```json id="we9n9m"
{
 "agents": {
 "defaults": {
```

```

 "model": "ollama-local/qwen2.5:14b"
 },
 "list": {
 "main": {
 "model": "ollama-local/qwen2.5:14b"
 },
 "code-operator": {
 "model": "ollama-local/qwen2.5-coder:7b"
 }
 }
}
}
}

```

The exact schema should follow the OpenClaw sub-agent documentation and your current `openclaw.json`.

Before editing, always back up the current config.

## Update the Sandbox Config

If working from the host, first export the current sandbox config:

```
```bash id="9j5hbo" export SANDBOX=my-assistant export DOCKER_CTR=openshell-cluster-nemoclaw
```

```
docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
cat /sandbox/.openclaw/openclaw.json > /tmp/openclaw.json
```

Edit the local file:

```
```bash id="gmeabx"
cp /tmp/openclaw.json /tmp/openclaw.updated.json
nano /tmp/openclaw.updated.json
```

Upload the updated config:

```
```bash id="2q7eau" docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
chmod 644 /sandbox/.openclaw/openclaw.json
```

```
docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
chmod 644 /sandbox/.openclaw/.config-hash
```

```
cat /tmp/openclaw.updated.json | docker exec -i "$DOCKER_CTR" kubectl exec -i -n openshell
"$SANDBOX" -c agent --
sh -c 'cat > /sandbox/.openclaw/openclaw.json'
```

```
docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
/bin/bash -c "cd /sandbox/.openclaw && sha256sum openclaw.json > .config-hash"
```

Optionally lock the config after updating:

```
```bash id="v8iho2"
docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
 chmod 444 /sandbox/.openclaw/openclaw.json

docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
 chmod 444 /sandbox/.openclaw/.config-hash
```

## Check Gateway Logs

After uploading the config, check whether OpenClaw reloaded correctly:

```
```bash id="atjex0" docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
tail -n 100 /tmp/gateway.log
```

Look for provider reload, agent list reload, or config validation errors.

Add Sub-Agent Credentials

For Ollama, you usually do not need real credentials.

If OpenClaw requires an auth profile, create one for the sub-agent using a dummy key:

```
```text id="35t6ip"
/sandbox/.openclaw/agents/code-operator/agent/auth-profiles.json
```

Example conceptual content:

```
```json id="sqn6q0" { "ollama-local": { "apiKey": "dummy" } }
```

Then fix ownership:

```
```bash id="sxccgt"
docker exec "$DOCKER_CTR" kubectl exec -n openshell "$SANDBOX" -c agent --
 chown -R sandbox:sandbox /sandbox/.openclaw/agents/code-operator
```

## Allow Ollama Network Access

If Ollama runs outside the sandbox, the sandbox must be allowed to reach it.

Common endpoints:

```
```text id="my4g68" http://host.docker.internal:11434 http://127.0.0.1:11434 http://<host-ip>:11434
```

Which one works depends on where NemoClaw, OpenShell, and Ollama are running.

If the request is blocked by OpenShell network policy, approve or add an egress rule for the binary making the request. For OpenClaw gateway requests, this is often:

```
```text id="vudx7r"
/usr/local/bin/node
```

Allow access to the Ollama endpoint and port:

```
```text id="wmiq5s" host: <ollama-host> port: 11434
```

Add Delegation Instructions

Create a task instruction file in the shared workspace:

```
```text id="0nl9pr"
/sandbox/.openclaw/workspace/TOOLS.md
```

Example:

```
```markdown id="slzt2y"
```

Task-Specific Delegation Instructions

When the user asks for code analysis, debugging, or command review, delegate the task to the `code-operator` sub-agent using `sessions_spawn`.

Use `/sandbox/.openclaw/workspace/` for files shared between agents.

The primary agent should: - understand the user's goal - prepare relevant files or logs in the workspace - call `sessions_spawn` with the task and file paths - review the sub-agent result before answering the user

The `code-operator` sub-agent should: - read files only from the provided workspace paths - focus on code, logs, commands, and configuration - return concise findings and recommended fixes

OpenWebUI Role

OpenWebUI can still be used for:

```
```text id="59bmnw"
chatting with the same Ollama models
testing prompts
```

```
checking model behavior
building a NemoClaw knowledge collection
```

But OpenWebUI does not automatically read or enforce:

```
```text id="4svh8x" sessions_spawn agents.list OpenClaw tool policy NemoClaw sandbox paths per-agent
auth profiles
```

Those remain OpenClaw/NemoClaw responsibilities.

Recommended Setup

Use this layout:

```
```text id="5g7f32"
```

Ollama:

Runs local models.

OpenWebUI:

Connects to Ollama for manual chat and testing.

NemoClaw/OpenClaw:

Connects to Ollama for agent execution.

Uses openclaw.json for agents.list and sub-agents.

Uses sessions\_spawn for delegation.

This keeps OpenWebUI simple and lets NemoClaw/OpenClaw handle agent orchestration.